

1. Executive Summary

F365 Advanced is a cross-platform Expo / React Native health and fertility app built in Rork. It runs on iOS, Android, and web through Expo Router and React Native Web. The current stack combines a React Native frontend, Supabase authentication/database, Hono + tRPC backend routes, Rork toolkit services, Mixpanel analytics, Stripe hosted checkout, Expo notifications, and fertility/AI modules.

2. Current Runtime and Project Structure

App root: expo/

Main entry: expo-router/entry

Root layout: expo/app/_layout.tsx

Navigation: Expo Router Stack plus tabs under app/(tabs)

Package manager: bun

Framework versions: Expo SDK 54, React Native 0.81.5, React 19.1.0, Expo Router 6.0.15

Backend style: Hono + tRPC, with React Query provider at app root

Database/Auth provider: Supabase

Analytics: Mixpanel custom HTTP implementation

Payments: Stripe hosted checkout links and subscription tables

Key files:

- expo/app/_layout.tsx: root providers, session restore, country access check, stack routes
- expo/constants/feature-keys.ts: feature flag key list including fertility_pathway, cost_estimator, ivf_roadmap, doctor_questions
- expo/lib/supabase.ts: Supabase client configuration
- expo/lib/trpc.ts: tRPC client wiring
- expo/store/user-store.ts: profile, session/user data, login/logout/register state
- expo/store/feature-gate-store.ts: country feature flag loading
- expo/utils/country-access.ts: country_access check on app launch
- expo/utils/mixpanel.ts: analytics event transport
- expo/docs/DEVELOPER-HANDOVER.md: existing long technical handover
- expo/docs/AWS_MIGRATION_CHECKLIST.md: existing migration/compliance checklist

3. Feature List

Core app features:

- Login, registration, onboarding, reset password, consent management
- Cycle dashboard, period/fertility calendar, daily log entry, daily check-in
- Mood tracking, symptoms, flow, discharge, pain, cravings, sexual activity, lifestyle notes
- Insights, advanced analytics, cycle charts, mood charts, irregularity alerts
- AI health chatbot, AI insights, AI mood forecast, prediction confidence badges
- Data export, privacy policy, terms of service, help/support, reminders
- Subscription/paywall, subscription management, redeem code flow
- Telehealth screens: doctor directory, appointment booking, appointments, consultation

UI

- Pregnancy/postpartum modules and stores
- Partner sharing module, partner link, partner summary, education, relationship dashboard
- Emergency contacts and safety UI

Fertility pathway features currently routed:

- app/fertility-pathway.tsx: Fertility Pathway hub
- app/cost-estimator.tsx: IVF/fertility treatment cost estimator
- app/ivf-roadmap.tsx: IVF journey roadmap
- app/doctor-questions.tsx: doctor question generator
- app/fertility-predictions.tsx: fertility predictions
- app/pcos-insights.tsx: PCOS insights

Feature flag keys to manage from admin dashboard:

- fertility_pathway
- cost_estimator
- ivf_roadmap
- doctor_questions
- fertility_predictions
- pcos_insights
- plus all core keys in constants/feature-keys.ts

4. Country Access and Feature Gating

The app should not use separate admin UIs for feature control. Admin control is handled externally through the country_feature_flags and country_access tables.

country_access:

Purpose: block/allow complete app access by country.

Current app behavior:

- Pre-login: app checks detected device country in app/_layout.tsx through utils/country-access.ts
- Post-login: app checks the user's profile country after session restore
- No row means allowed
- Row with is_enabled=false means blocked and shows CountryBlockedScreen

country_feature_flags:

Purpose: enable/disable app features per country and access tier.

Current app behavior:

- Feature keys are defined in constants/feature-keys.ts
- FeatureGate component can grey out or block disabled features
- Admin dashboard manages rows externally
- No separate mobile admin UI should be built

Recommended feature rows:

- AU / fertility_pathway / enabled true
- AU / cost_estimator / enabled true
- AU / ivf_roadmap / enabled true

- AU / doctor_questions / enabled true
- IN, PK, US, UK rows as needed by rollout policy

5. Database Tables and Data Ownership

Main Supabase tables referenced by the code/docs:

- users
- symptom_logs
- custom_symptoms
- mood_logs
- lifestyle_logs
- predictions
- ai_insights
- ai_predictions
- redemption_codes
- subscriptions
- subscription_orders
- cancellation_requests
- daily_checkins
- sexual_activity_logs
- partner_education_progress
- reminder_preferences
- country_feature_flags
- country_access
- ivf_cost_estimates
- doctor_question_templates
- user_feedback

New fertility/support tables:

ivf_cost_estimates:

- id uuid primary key
- country_code text
- treatment_type text, examples: ivf_standard, ivf_icsi, iui, egg_freezing
- min_cost numeric
- max_cost numeric
- currency text
- notes text nullable
- source text nullable
- is_active boolean
- created_at timestampz
- updated_at timestampz

Admin purpose: seed AU, IN, PK, US, UK treatment cost ranges.

Mobile purpose: cost-estimator screen reads active rows for the user's country.

doctor_question_templates:

- id uuid primary key
- category text
- treatment_type text nullable

- country_code text nullable
- question text
- display_order integer
- is_active boolean
- created_at timestamptz
- updated_at timestamptz

Admin purpose: maintain reviewed fertility/IVF question templates.

Mobile purpose: doctor-questions screen prefers DB templates and falls back to built-in questions if empty.

user_feedback:

- id uuid primary key
- user_id uuid/text nullable depending current schema
- screen text nullable
- feedback_type text
- message text
- metadata jsonb nullable
- created_at timestamptz

Admin purpose: view app/user feedback.

Mobile purpose: ReportFeedback component inserts reports.

No fertility_pathway_config table is needed. Use country_feature_flags with keys fertility_pathway, cost_estimator, ivf_roadmap, doctor_questions.

6. Authentication

Provider: Supabase Auth.

Supported flow:

- App launch restores Supabase session with timeout
- If session exists, fetch users row by auth_id
- If onboarded, route to /(tabs)
- If not onboarded, route to /onboarding
- If no session, show /login
- Logout calls Supabase signOut and clears local state

Important file areas:

- app/_layout.tsx: restore session and routing
- app/login.tsx: email/password login
- app/onboarding.tsx: registration/profile collection
- app/reset-password.tsx: password reset
- store/user-store.ts: user profile and auth-related state

Known consideration: demo mode can create local-only user state. Server/database writes must handle missing Supabase auth gracefully.

7. AI Integrations

Current AI capabilities:

- AI Chatbot: conversational health assistant through tRPC ai.chat route and Rork toolkit generation path
- AI Insights / Predictions: cycle and fertility-related guidance/prediction logic
- AI Mood Forecast: mood prediction timeline and explanations
- Symptom Checker: guided symptom assessment with safety disclaimer patterns

AI guardrails implemented:

- Medical disclaimers
- Crisis/safety footer where applicable
- Consent gate for AI processing
- Safe fallback responses
- Confidence badges
- Input validation in backend routes
- User-friendly failures instead of crashing

Future AI scaling options:

- Replace Rork toolkit calls with direct OpenAI, Vercel AI Gateway, or AWS Bedrock
- Add server-side AI moderation/content filtering
- Pin model versions
- Add AI response audit logs
- Add cost controls/rate limits
- Add retrieval-augmented health knowledge base using OpenSearch/Pinecone plus a reviewed medical content source

8. API / Backend Connections

Current backend pattern:

- Hono mounted under API routes
- tRPC used for typed backend calls
- React Query provider wraps the app
- Supabase direct client access exists in several stores for database operations

External connections:

- Supabase: auth + Postgres
- Rork: project runtime/toolkit/API environment
- Mixpanel: analytics event HTTP API
- Stripe: hosted checkout/subscription data
- Expo Notifications: local scheduled reminders and notification registration
- Mail composer / clipboard / print / sharing where screens use Expo modules

Configured environment variables visible in Rork:

- EXPO_PUBLIC_SUPABASE_URL
- EXPO_PUBLIC_SUPABASE_ANON_KEY
- EXPO_PUBLIC_MIXPANEL_TOKEN
- EXPO_PUBLIC_RORK_DB_ENDPOINT
- EXPO_PUBLIC_RORK_DB_NAMESPACE
- EXPO_PUBLIC_RORK_DB_TOKEN
- EXPO_PUBLIC_RORK_AUTH_URL
- EXPO_PUBLIC_RORK_API_BASE_URL

- EXPO_PUBLIC_TOOLKIT_URL
- EXPO_PUBLIC_PROJECT_ID
- EXPO_PUBLIC_TEAM_ID
- Stripe Secret Key
- Stripe Publishable Key
- F365 test api

Developer note: public Expo env vars are bundled into the client. Never put server-only secrets in EXPO_PUBLIC variables.

9. Payments / Stripe

Current approach:

- Hosted Stripe checkout links rather than native embedded payments
- Subscription tables exist for user subscription records, orders, cancellation requests, refund fields
- Subscription store manages local state and validation logic

Production needs:

- Stripe webhook handler for checkout/session/subscription events
- Webhook signature verification
- Server-side subscription status sync
- Live price IDs/links when moving out of test mode

10. Analytics / Mixpanel

Current approach:

- Custom HTTP Mixpanel service in utils/mixpanel.ts
- Tracks app opened, login/register/logout, profile updates, subscriptions and feature usage events
- User properties are set after login/register

Production needs:

- Ensure token is sourced from EXPO_PUBLIC_MIXPANEL_TOKEN
- Respect consent toggles
- Avoid sending health detail content; track metadata/counts only

11. AWS / Future Scaling Migration

Likely AWS target architecture:

- Frontend: Expo app stays as-is
- API: Hono/tRPC deployed to Lambda + API Gateway, or ECS if long-running server preferred
- Database: Supabase Postgres migrated to RDS PostgreSQL or Aurora Serverless
- Auth: Supabase Auth migrated to Amazon Cognito
- Secrets: AWS Secrets Manager and SSM Parameter Store
- Encryption: KMS, RDS encryption, server-side AES-256-GCM for sensitive data
- Logs: CloudWatch structured logs
- Audit archive: S3 with lifecycle/retention policies
- Push: SNS plus FCM/APNs if server-side push is required
- WAF/Rate limit: AWS WAF + API Gateway throttling

Migration priority:

1. Move hardcoded secrets/tokens to env/config where applicable
2. Add backend auth middleware and server-side authorization
3. Build Stripe webhook handler
4. Verify delete/export covers all user tables
5. Migrate database schema/data using pg_dump/pg_restore style process
6. Replace Supabase Auth with Cognito if AWS-only target is required
7. Replace direct Supabase client writes with API-layer writes
8. Add monitoring, rate limiting, audit persistence, test coverage

12. Web Page / Website Connection Notes

The app currently supports React Native Web preview through Expo/Rork. If connecting to a marketing/admin web page:

- Keep admin dashboard separate from mobile app
- Admin dashboard should manage Supabase tables directly or via protected edge functions
- Mobile app should only read feature flags and active public/reference rows
- Do not expose service role keys to mobile or public web clients
- Use edge functions/server API for admin mutations

13. Known Technical Debt / Developer Attention

- Some docs mention older hardcoded credentials; verify current source before launch
- Some pregnancy module screens may still be partial/stub-style compared with stores
- Telehealth appears mock-data oriented unless connected to a real provider API
- Stripe webhook is required for production-grade subscription status
- AI moderation and audit logs are not yet production-grade
- Supabase RLS must be reviewed for every new table
- Rork toolkit calls need replacement if migrating fully away from Rork
- Test coverage is limited or absent and should be added before scale

14. Handover Checklist for Developer

- Read expo/docs/DEVELOPER-HANDOVER.md
- Read expo/docs/AWS_MIGRATION_CHECKLIST.md
- Read expo/docs/COUNTRY_FEATURE_FLAGS.md
- Read expo/docs/EMAIL_ENCRYPTION.md
- Review app/_layout.tsx session and country access logic
- Review constants/feature-keys.ts for all feature flags
- Review lib/supabase.ts and all store imports of supabase
- Confirm Supabase schemas for ivf_cost_estimates, doctor_question_templates, user_feedback, country_feature_flags, country_access
- Seed ivf_cost_estimates for AU, IN, PK, US, UK
- Seed doctor_question_templates
- Configure country_feature_flags by country; do not create fertility_pathway_config
- Verify user_feedback insert works from ReportFeedback
- Verify app behavior in Expo web preview and on Expo Go

15. Suggested Rork Prompt for Future Technical Extraction

Ask Rork:

Please audit this Rork Expo project and generate a developer handover covering: all screens/features, database tables and fields, Supabase auth/database usage, country_feature_flags and country_access logic, AI integrations, Stripe/payment flow, Mixpanel analytics, environment variables, Rork-specific dependencies, API/backend routes, AWS migration path, scaling risks, and production readiness checklist. Include exact file paths and mark what is implemented vs pending.